

The Sandbox Add-on CTA, end to end

10 commits, 36 files, +2186/−117 — a field guide to what exists, what changed, and why the Kotlin reads the way it does.

The feature in one sentence: on the sandbox violations dashboard, a non-entitled org's preview banner now carries a real action — **org admins enable the sandboxing add-on** in place, and **non-admin members request it**, which emails every org admin at most once per 48-hour window.

You're viewing sample data for the sandboxing add-on

Sample data

Sandboxing runs each task in a sandbox to catch unexpected file reads and writes... [Docs](#)

[Go to add-ons page](#)

Enable add-on

admin → Enable add-on

member → Request add-on

request active → Requested (disabled, tooltip: who + when)

provisioning → Enabling... (no action)

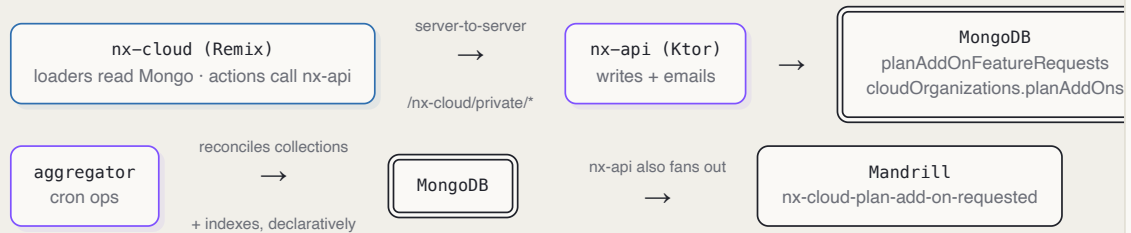
gated off → plain link (old behavior)

1. The system you're standing in
2. Flow 1: a member requests
3. Flow 2: an admin enables
4. Kotlin, layer by layer
5. Arrow's Raise, a crash course
6. The atomic claim
7. Wiring and "migrations"
8. The TypeScript half
9. Why this shape — the house rules

1 The system you're standing in

Ocean runs three services that matter here. The unusual part, if you're coming from single-backend apps: **the Remix app talks to MongoDB directly for reads**, while writes that carry business rules go through nx-api. This PR follows that split exactly — TypeScript reads the new collection, Kotlin is its only writer.

WHO TOUCHES WHAT



The gateway (cloud-infrastructure) routes `PathPrefix /nx-cloud/private` to a 404 backend before the general `/nx-cloud` match — so nx-api's "private" routes are unreachable from the internet. That's why the Kotlin handler can be `Unprotected`: the authenticated Remix action *is* the authorization boundary.

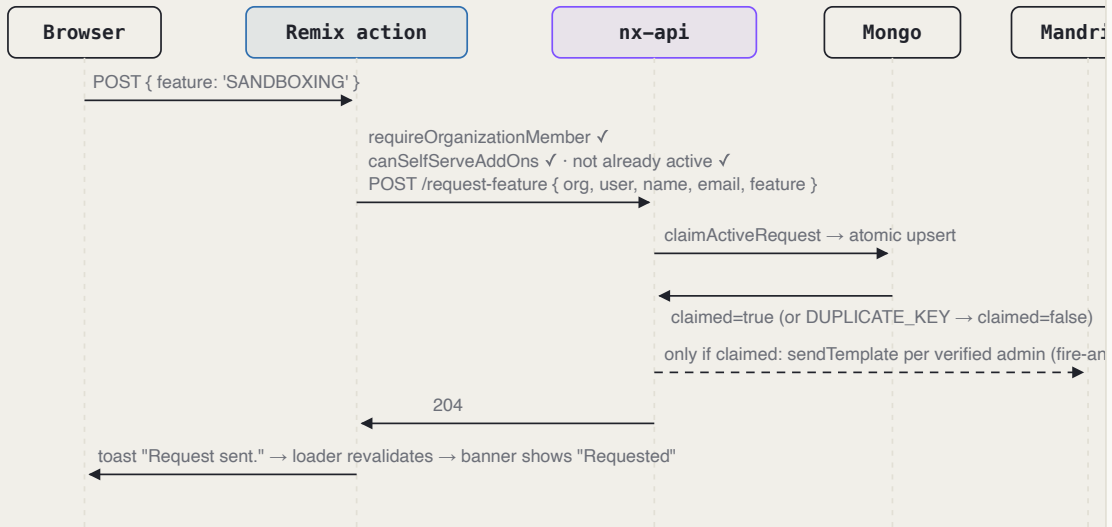
New pieces this PR adds, by service:

SERVICE	NEW	REUSED
KOTLIN nx-api	POST <code>/plan-add-on/v1/request-feature</code> → <code>PlanAddOnFeatureRequestService</code> → <code>PlanAddOnFeatureRequestRepository</code> (atomic claim) → email fan-out	<code>PlanAddOnNotificationService</code> plumbing, <code>PlanAddOnProvisionError</code> , the whole provision flow
KOTLIN aggregator	one line: a unique index in <code>indexConfiguration</code>	the reconcile machinery
TS nx-cloud	<code>SandboxAddOnCta</code> , loader CTA state, <code>RequestPlanAddOnAction</code> , <code>EnablePlanAddOnAction</code> , <code>canSelfServeAddOns</code>	<code>submitPlanAddOnSelection</code> (extracted from the add-ons form's action), <code>ConfirmationDialog</code> , pricing constants

2 Flow 1: a member requests the add-on

The interesting property: **no matter how many members click, or how fast, admins get at most one email per 48 hours**. Everything below exists to make that sentence true under concurrency.

MEMBER REQUEST — SEQUENCE



Everyone in the org sees the same "Requested" state afterwards, because the loader re-reads the org-wide document on every visit — persistence *is* the refresh-survival mechanism, no client state involved.

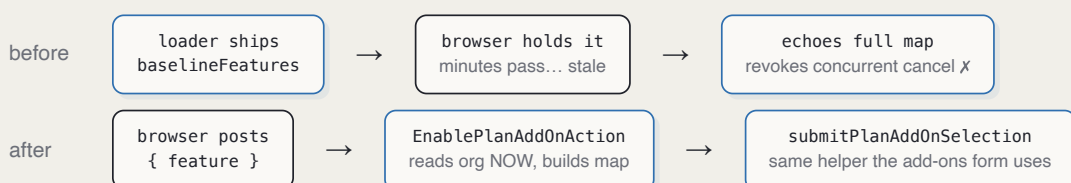
3 Flow 2: an admin enables

The admin path reuses the machinery the add-ons page already had. Two things are specific to this PR:

The cluster comes along. The dashboard's entitlement check (`isSandboxingAvailableForOrganization`) requires *both* `DEDICATED_COMPUTE_CLUSTER` and `SANDBOXING`. Enabling sandboxing alone would never light the page up. So the enable action always pulls the cluster in, and the dialog discloses the \$99/mo committed spend when the cluster is new.

The browser sends one field. An earlier revision shipped the org's active-feature list to the client, which echoed it back at submit (the provision endpoint treats its body as *complete desired state* — absent features count as cancellations). Review caught the flaw: between page load and click, another admin's cancellation would be silently revoked by the stale echo. Now the client posts only `{ feature }` and the server computes the full map at submit time:

STALE-ECHO FIX — BEFORE VS AFTER

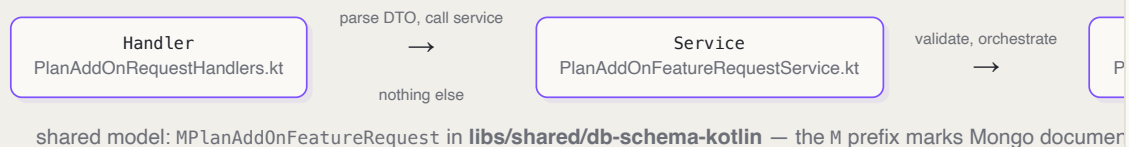


Downstream, Kotlin's `PlanAddOnProvisionService` branches on whether infrastructure work is needed: a first-time cluster fires the provision-tenant webhook (an Argo workflow in the ops cluster), records a `REQUESTED` row, and completes minutes later via callback — the banner shows "c Enabling..." and the page polls every 10s until the entitlement lands. If the cluster already exists, sandboxing reconciles onto the org document immediately.

4 Kotlin, layer by layer KOTLIN

nx-api is a Ktor server with strict, boring layering — the same four roles every entity gets. The house rule (enforced in review, documented in `.claude/skills/altan-review`): handlers parse and delegate, services own business logic, repositories own persistence, models are dumb data.

THE REQUEST-FEATURE SLICE



`libs/shared/db-schema-kotlin/src/main/kotlin/PlanAddOns.kt`

```
@Serializable // ① kotlinx.serialization: BSON codec
data class MPlanAddOnFeatureRequest(
    @Contextual val _id: ObjectId = ObjectId(), // ② non-@Serializable types need a c
    @Contextual val organizationId: ObjectId,
    val feature: PlanAddOnFeature, // ③ enum, serialized by name – match
    @Contextual val requestedByUserId: ObjectId,
    val requestedByName: String, // ④ denormalized: UI shows "who aske
    val requestedByEmail: String, // and the row stays meaningful if
    @Contextual val createdAt: Date = Date(),
) {
    companion object { // ⑤ Kotlin's "static" home: constant
        const val COLLECTION_NAME = "planAddOnFeatureRequests"
        const val ACTIVE_WINDOW_HOURS = 48 // ⑥ mirrored in TS model-db; both sides
    }
}
```

A `data class` is Kotlin's record: `equals` / `hashCode` / `copy` for free, immutable `val` fields. The double bookkeeping in ⑥ is deliberate — Kotlin enforces the window on write (the claim), TS reads it for display (when the "Requested" chip clears) — and each side's comment points at the other.

`apps/nx-api/.../handlers/planaddonrequests/PlanAddOnRequestHandlers.kt`

```

route("/nx-cloud/private/plan-add-on/v1") {
    with(PlanAddOnProvisionErrorHttpMapper) { // ① scoped receiver: maps typed er
        with(Unprotected) { // ② no auth — safe because the LB
            if (featureRequestService != null) { // ③ nullable service = feature gat
                raisingPost("/request-feature") { call, _ ->
                    val request = receiveOrRaise<PlanAddOnFeatureRequestBody>(call)
                    featureRequestService.requestFeature(/* ...fields... */)
                    SuccessResponse.NoContent // ④ that's the whole handler. No D
                }
            }
        }
    }
}

```

The `with(X) { }` nesting is Kotlin's *context receiver* style: it puts `X` in scope so functions like `raisingPost` can find the error mapper and auth policy without threading parameters. `raisingPost` is the codebase's blessed handler shape — it runs the block inside an Arrow error scope and converts anything `raise`d into the mapped HTTP response. Which brings us to Arrow.

5 Arrow's Raise, a crash course KOTLIN

nx-api doesn't throw exceptions for expected failures. It uses [Arrow's Raise](#): errors are values, declared in the function signature, and the compiler tracks who must handle them. If you know Rust's `Result` or Haskell's `Either` — this is that, with nicer ergonomics.

	EXCEPTIONS	RAISE
signature	<code>fun find(): Widget</code> (failure invisible)	<code>context(raise: Raise<WidgetError>) fun find(): Widget</code>
fail	<code>throw WidgetException()</code>	<code>raise(WidgetError.NotFound(id))</code>
guard	<code>if (x == null) throw ...</code>	<code>ensureNotNull(x) { Error(...) }</code> (smart-casts x, too)
consume	<code>try / catch</code>	<code>either { find() }.fold(onLeft, onRight)</code> — or let <code>raisingPost</code> do it

The pattern in every repository/service method in this PR:

```

context(raise: Raise<PlanAddOnProvisionError>) // ① "this can fail with this erro
suspend fun requestFeature(...): MPlanAddOnFeatureRequest = with(raise) { // ② brings
    ensure(ObjectId.isValid(organizationId)) { // ③ guard: raises instead of retu
        PlanAddOnProvisionError.InvalidOrganizationId(organizationId)
    }
    ...
}

```

Two gotchas the codebase's `arrow-kt-patterns` skill calls out, both of which shaped this PR:

a Real exceptions still exist at the driver boundary. The Mongo driver throws `MongoException`; the repository catches it and converts to a raised `DatabaseError`. That's the sanctioned direction: catch at the boundary, raise inward.

b Don't `raise` inside a `try`. `raise` is implemented with a control-flow exception, so a careless `catch` can swallow it. The claim method (next section) was restructured in review so the `try` contains only the driver call, and all raising happens outside it.

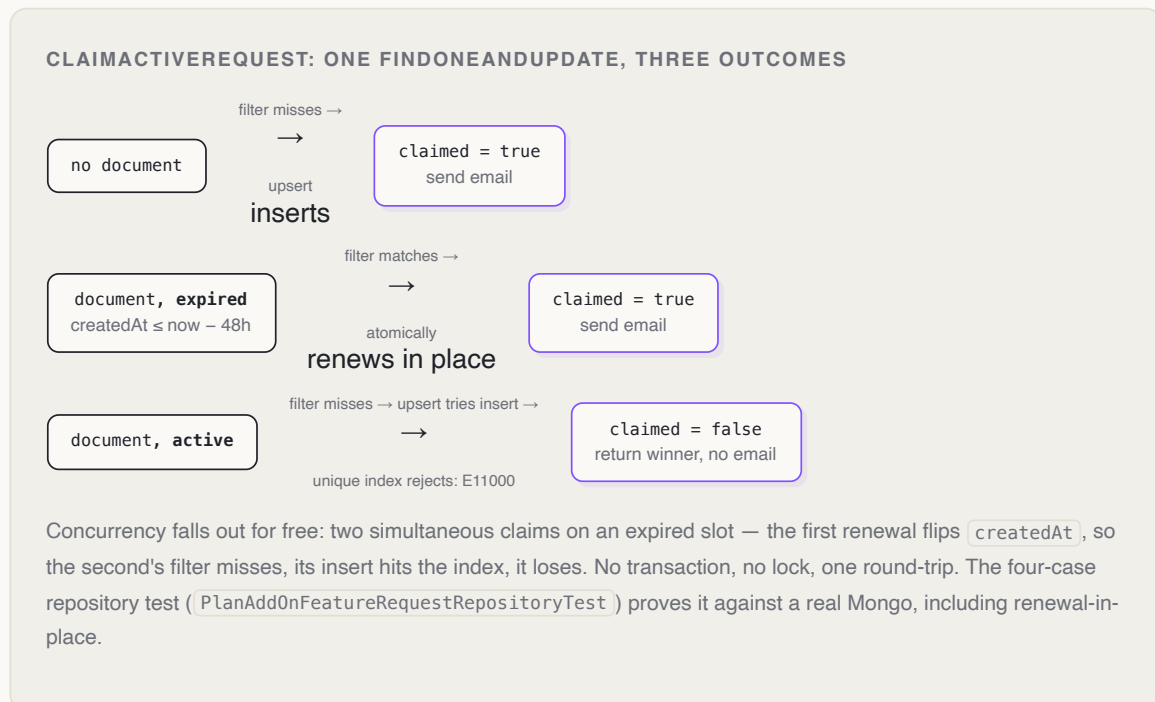
Tests wrap calls in `either { }`, which reifies the outcome as a value:

`result.isRight()` for success, `assertIs<SomeError>(result.leftOrNull())` for failure — no `assertThrows`.

6 The atomic claim — the one clever bit KOTLIN

First version was `findActive()` then `insert()`. Review flagged the classic TOCTOU race: two concurrent requests both see "no active row," both insert, admins get two emails — exactly what the cooldown exists to prevent (Altan's original words: "otherwise we'll hear all about it during our pen tests").

The fix changes the data model instead of adding locks: **one document per org+feature**, **ever**, enforced by a unique index on `{organizationId, feature}`, renewed in place after expiry. Then a single conditional upsert sorts every case:



`apps/nx-api/.../persistence/PlanAddOnFeatureRequestRepository.kt` (condensed)

```

val claimed = try {
    collection.findOneAndUpdate(
        and(
            MPlanAddOnFeatureRequest::organizationId eq req.organizationId, // typed
            MPlanAddOnFeatureRequest::feature eq req.feature, // never
            MPlanAddOnFeatureRequest::createdAt lte windowStart(), // only an EXPIRED
        ),
        combine(/* $set requester fields + createdAt = now */),
        FindOneAndUpdateOptions().upsert(true).returnDocument(AFTER),
    )
} catch (e: MongoException) {
    if (ErrorCategory.fromErrorCode(e.code) == ErrorCategory.DUPLICATE_KEY) null // a
    else raise(PlanAddOnProvisionError.DatabaseError("claim plan add-on feature request"))
}
if (claimed != null) return@with PlanAddOnFeatureRequestClaim(claimed, claimed = true)
val existing = ensureNotNull(findActive(...)) { /* DatabaseError */ } // surface the w
PlanAddOnFeatureRequestClaim(existing, claimed = false)

```

The service layer then reads almost like the requirement: claim; if you didn't win, hand back the winner; if you did, notify — where the email is `runCatching`-wrapped fire-and-forget, because a Mandrill hiccup must never fail a member's request. Window boundaries are exactly complementary across languages: Kotlin renews at `createdAt ≤ now-48h`, both sides treat `createdAt > now-48h` as active — no instant where a request is both.

7 Wiring and "migrations" KOTLIN

No DI framework — `Main.kt` is one long function doing manual constructor injection. Idiom worth noticing: **a nullable service is a feature gate**. `provisionService` is null when the ops webhook env vars aren't configured (on-prem installs), and the handler simply doesn't mount those routes. The new `featureRequestService` needs no ops client, so it mounts independently — that's why the handler takes two nullable services and registers routes conditionally inside one `route` block.

Schema management is declarative, not migration scripts. Mongo creates collections on first write; indexes come from a single map the aggregator reconciles on its migrations pass:

```
apps/aggregator/.../migrations/ReconcileCollectionsAndIndices.kt
```

```

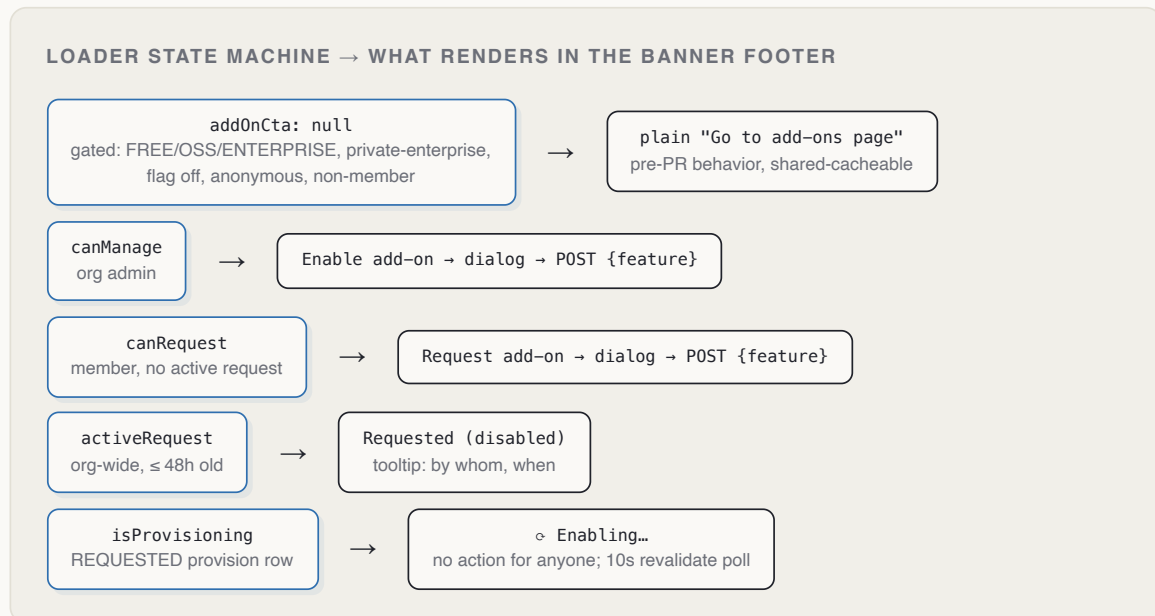
MPlanAddOnFeatureRequest.COLLECTION_NAME to listOf(
    parseIndexConfig("organizationId:1,feature:1", unique = true), // the claim's cor
),

```

The one deploy-order note: the atomic claim depends on that unique index existing. The surface is PostHog-flagged, so the safe sequence is: deploy → let one aggregator migrations pass run → open the flags. (Verified: cloud-infrastructure manages no Mongo indexes and needs no changes at all for this PR.)

8 The TypeScript half TS

One component, one loader field, three server actions. `SandboxAddOnCta` lives in `feature-analytics` (not the bulk-deletable `feature-add-on-previews` — dependency direction: previews already imports analytics) and reads its state from the route loader:



Three details that came out of review, worth internalizing:

a One gate, three call sites. The "can this org self-serve add-ons at all" check (flag ∨ env, plan, deployment mode) is `canSelfServeAddOns()` in `util-flag-foundry` — used by the loader (shows the button), the request action, and the enable action (authorize it).

Loader/action pairs must share gates or they drift: member sees a button, action 403s.

b Caching is part of the contract. This loader used to serve `max-age=3600` — fine for org-shaped data, wrong the moment the response says who *you* are. Responses carrying CTA state switch to `no-store`; gated-off and entitled responses stay shared-cacheable.

c Constants over prose. The dialog interpolates

`DEDICATED_COMPUTE_CLUSTER_PRICE_USD` / `_INCLUDED_CREDITS` from model-organizations (moved there from the add-ons feature lib) instead of hardcoding "\$99" — a price change can't silently lie in one surface.

9 Why this shape — the house rules it satisfies

Everything above maps to written-down convention (`.claude/skills/altan-review`, distilled from ~240 reviewed PRs, and `arrow-kt-patterns`):

RULE	WHERE THIS PR OBEYS IT
Handlers are thin	<code>request-feature</code> handler: parse DTO → service → 204. Zero logic, zero Mongo.
Repositories own persistence only	Service constructs <code>MPlanAddOnFeatureRequest</code> ; repository only claims/reads. One repo per entity.
Typed errors, never throw	<code>Raise<PlanAddOnProvisionError></code> end to end; driver exceptions converted at the boundary.
Handle unique-index collisions with Raise semantics	<code>DUPLICATE_KEY</code> isn't an error here — it's the business answer ("someone already asked").
Typed property references in queries	<code>MPlanAddOnFeatureRequest::createdAt lte ...</code> , never string field names.
<code>find().limit(1)</code> over <code>countDocuments()</code>	<code>findActive</code> — existence checks shouldn't scan.
Tests describe behavior, minimal mocking	Repo test runs the four claim states against real Mongo; no ticket numbers in names.
Derive security state server-side	Requester identity from the session; desired feature state computed at submit (the stale-echo fix).
Email failures are non-blocking, logged as errors with IDs	<code>notifyMemberRequested</code> : <code>runCatching</code> + structured log, request never fails on Mandrill.

The one-paragraph takeaway for the Kotlin side: nx-api is Ktor + manual DI + strict handler/service/repository layering, with Arrow's Raise replacing exceptions so failure modes live in function signatures. Mongo access is typed and declarative — property-reference queries in repositories, indexes reconciled from one map by the aggregator. When you need concurrency guarantees, the house move is to lean on Mongo's atomic primitives plus a unique index rather than locks — and to prove it with a repository test against the real database.

Companion docs: plan & decisions in `dot_ai/2026-07-02/tasks/q-520-sandbox-dashboard-add-on-toggle.md` ,
Mandrill template draft in `q-520-mandrill-template-draft.md` , screenshots in `q520-shots/` . PR: nrwl/ocean #12211.